

DataLab 实验报告

姓名：韩滨泽 学号：2025201818

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse	logicalShift	leftBitCount	float_i2f	floatScale2	float64_f2i	floatPower2
37.00	1.00	1.00	2.00	4.00	4.00	3.00	3.00	4.00	4.00	4.00	3.00	4.00

解题报告

亮点

- logtwo** —— 白名单只有 `>` `<` `>>` `<<` `|` (没有 `if`、没有 `&`、没有 `+`)，用"比较结果当开关"实现无分支二分定位最高位。
- leftBitCount** —— 涂抹 + SWAR popcount，全程无控制流、无比较，30/50 个符号。
- float_i2f** —— 手工实现就近偶数舍入与有效数进位，符号数刚好卡满 30/30。

12 题思路总览

题目	解题思路	符号数/上限
bitAnd	德摩根律: $x \& y = \sim(\sim x \mid \sim y)$	4/7
bitXor	拆成 $(x \mid y) \& \sim(x \& y)$ ，其中 $x \mid y$ 再用德摩根展开，符号数正好卡满	7/7
samesign	0 既不正也不负，用 <code>!</code> 单独处理；其余用 $(x \wedge y) \gg 31$ 比较符号位	6/12
logtwo	把比较得到的 0/1 当开关，按 16/8/4/2/1 逐轮二分定位最高位	18/25
byteSwap	取出第 n、m 个字节 → 清零原位置 → 移位回填	15/17
reverse	循环 32 次逐位反转；白名单没有比较运算符，用 <code>i</code> 本身当循环条件	5/30
logicalShift	算术右移后与掩码，掩码写法规避"移位 32 位"的未定义行为	6/20
leftBitCount	$\sim x$ 涂抹出全 1 前缀 + SWAR popcount，答案写成 $33 + \sim \text{popcount}$	30/50
float_i2f	无符号补码取绝对值 + 找最高位 + 就近偶数舍入 + 有效数进位处理	30/30
floatScale2	按阶码分四类改位域：NaN/Inf、0 与非规格化、exp=254、正常	11/30
float64_f2i	拼出 53 位有效数后右移截断，等价于向零舍入	16/60
floatPower2	规格化写阶码、非规格化写尾数、两端分别返回 0 与 +INF	9/30

logtwo

白名单只有 `>` `<` `>>` `<<` `|`，参考实现用的 `if`、`&` 和数组都不能用。思路分三点：

- 目标是最高位 1 的下标 $p \in [0, 31]$ ，它用二进制表示就是 5 位，所以 **5 次比较** 足以二分确定；
- 每轮维护不变量"进入步长 2^k 这一轮时 $v < 2^{(2k)}$ "。在这个前提下，"第 $k \sim 2k-1$ 位里有 1"**等价于** $v \geq 2^k$ ，于是参考实现里的位掩码退化成一次普通比较（阈值取 $2^k - 1$ ，因为白名单没有 `>=`）；
- 用"无分支三件套"消掉 `if`：比较结果本身就是 0/1，可以当开关；`<< k` 把它变成增量 2^k ；`|` 负责累加（16/8/4/2/1 的二进制位互不重叠，`|` 与 `+` 等价，而白名单里没有 `+`）。剥离动作 `v >>= s` 完成，`s = 0` 时就是空操作。

```
for k in {16, 8, 4, 2, 1}:
    s = (v > 2^k - 1) << k      # 条件成立 → 剥离 2^k 位
    r = r | s
    v = v >> s
return r
```

结果 `r` 的五个二进制位，就是五次比较的答案从高位到低位拼出来的。

leftBitCount

不许 `if`/`while`，全部用位运算。核心恒等式：**前导 1 的个数 = $32 - (\sim x$ 中最高位 1 的位置 + 1)**。

1. 令 $y = \sim x$: x 的前导 1 变成 y 的前导 0;
2. 涂抹: $y |= y >> 1/2/4/8/16$ 把最高位 1 以下全部变成 1, 此时 `popcount(y)` 恰好是"最高位位置 + 1" (移位量 $1+2+4+8+16 = 31$, 刚好覆盖整个字);
3. SWAR popcount: 用 `0x55555555 / 0x33333333 / 0x0F0F0F0F` 把相邻 1/2/4 位两两归并相加; 最后 $y + (y >> 8)$ 、 $y + (y >> 16)$ 累加各字节的计数, 因为每字节计数 ≤ 8 , 加法不会跨字节进位, 所以这两步不需要掩码;
4. 白名单没有减号, 答案写成 $33 + \sim y$ (即 $32 - \text{popcount}$)。

边界情况不需要特判: $x = 0 \rightarrow y = -1 \rightarrow \text{popcount } 32 \rightarrow \text{答案 } 0$; $x = -1 \rightarrow y = 0 \rightarrow \text{popcount } 0 \rightarrow \text{答案 } 32$ 。

float_i2f

分四步:

1. $x == 0$ 直接返回 0;
2. 符号位单独拆出, 绝对值用**无符号补码** $\sim ux + 1$ 求。若用有符号取负, `INT_MIN` 会溢出 (未定义行为), 而无符号补码能得到正确的 2^{31} ;
3. `while (!(ux >> e)) e--` 从 31 往下找到最高位 1 的位置 e , 于是阶码字段为 $e + 127$;
4. 规格化与舍入, 是本函数的主体:
 - $e \leq 23$ 时有效位数不超过 24 位, **一定可以精确表示**, 左移对齐即可, 无需舍入;
 - $e > 23$ 时低 $\text{shift} = e - 23$ 位必须丢弃, 按 IEEE 默认的**就近偶数舍入**处理: 把丢弃部分 `low` 与半值 `half` 比较, 大于则进位、小于则舍去、**恰好相等 (平局) 时进位与否取决于保留部分末位 `sig & 1`**, 从而把结果凑成偶数;
 - 舍入可能使有效数进位到 25 位 ($\text{sig} >> 24$), 此时右移一位并把 e 加一。典型例子是 `0x7FFFFFFF`: 舍入后变成 2^{31} , 位模式为 `0x4F000000`。

符号数刚好 30: 因为 `&&`、`||`、`?:` 都不在白名单里, "大于"和"平局"这两个条件没法写成一表达式, 只能先用两个 `if` 分别设置 `roundup`, 再统一相加, 这是最省符号的写法。

反馈/收获/感悟/总结

这次实验最大的收获是"把控制流翻译成位运算"的思路: 比较结果可以直接当 0/1 使用, 左移可以代替分支, 按位或可以代替累加。印象最深的两处是 `float_i2f` 的舍入——必须与 $(\text{float})x$ 的就近偶数舍入完全一致——以及 `leftBitCount` 的涂抹 + SWAR popcount。真正的难点不只在算法, 还在于**符号数上限**: 为了省一个符号, 有时必须把两个条件合并成一个 `roundup` 变量, 或者把减法写成 $33 + \sim y$ 。此外, 先读 `test.py` 的源码把合规口径弄清楚 (哪些算符号、哪些只在白名单里查) 比反复试错有效得多。

参考的重要资料

1. *Computer Systems: A Programmer's Perspective* (CS:APP) 第 2 章"信息的表示和处理"——IEEE 754 格式、舍入规则、位运算与补码。
2. Henry S. Warren, *Hacker's Delight* (2nd ed.) ——SWAR 分段计数 (popcount) 与最高定位位的位运算技巧。
3. 课程 PPT《数据的机器级表示》——单精度/双精度格式与规格化、非规格化的定义。
4. 实验仓库中的 `README.md`、`test.py` 与 `tests.c` ——合规性检查口径 (白名单、Max ops 统计方式) 与各题的参考语义。